



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации информационных технологий
(МОСИТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Тестирование и верификация программного обеспечения»

Практическое занятие № 2

*ИКБО-43-23 ОГАНЕСЯН Д. К.
КОЩЕЕВ М.И.
ГУТКОВИЧ А.А.
ИВАНОВ А.В.
УЛЗЕТУЕВ А.С.*

(подпись)

АССИСТЕНТ

ИЛЬИЧЕВ Г.П.

(подпись)

**ОТЧЕТ
ПРЕДСТАВЛЕН**

«17»__ОКТЯБРЯ_2025 Г.

Москва 2025 г.

СОДЕРЖАНИЕ

1 ЦЕЛЬ И ЗАДАЧИ ПРАКТИЧЕСКОЙ РАБОТЫ.....	3
2 ПРАКТИЧЕСКАЯ ЧАСТЬ.....	4
2.1 Разработка модулей.....	4
2.2 Модульное тестирование.....	7
2.3 Мутационное тестирование.....	14
3 АНАЛИЗ И ВЫВОДЫ.....	19
3.1 Оценка качества тестирования.....	19
3.2 Анализ недостатков и их устранение.....	20
3.3 Итоговые выводы по результатам работы.....	20

1 ЦЕЛЬ И ЗАДАЧИ ПРАКТИЧЕСКОЙ РАБОТЫ

Цель работы: познакомить студентов с процессом модульного и мутационного тестирования, включая разработку, проведение тестов, исправление ошибок, анализ тестового покрытия, а также оценку эффективности тестов путём применения методов мутационного тестирования.

Для достижения поставленной цели работы студентам необходимо выполнить ряд задач:

- изучить основы модульного тестирования и его основные принципы;
- освоить использование инструментов для модульного тестирования (pytest для Python, JUnit для Java и др.);
- разработать модульные тесты для программного продукта и проанализировать их покрытие кода;
- изучить основы мутационного тестирования и освоить инструменты для его выполнения (MutPy, PIT, Stryker);
- применить мутационное тестирование к программному продукту, оценить эффективность тестов;
- улучшить существующий набор тестов, ориентируясь на результаты мутационного тестирования;
- оформить итоговый отчёт с результатами проделанной работы.

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Разработка модулей

Описание функциональности: Модуль представляет собой консольное приложение для конвертации единиц измерения, которое включает главное меню с возможностью конвертации длины, массы и температуры, просмотра истории операций и статистики; предоставляет пользователю интерфейс для выбора типа конвертации, ввода значений и единиц измерения, отображает результаты преобразований и сохраняет их в историю; содержит подменю для работы с историей (просмотр последних конвертаций, всей истории, статистики и очистка), а также механизмы валидации входных данных и обработки ошибок при выполнении конвертаций.

Листинг 5 — Исходный код.

```
import time
import core.convert_script as conversions
import core.validator as validator
import core.unit_formatter as unit_formatter
import core.history as history

def startup():
    print(r"""
/$$$$$\  $$$$$\  $$\  $$\  $$\  $$$\  $$$$$$\  $$$$$$\  $$$$$$\  $$$
$$$$$\ 
$$  _$$\  $$  _$$\  $$$\  $$ |$$ |  $$ |$$  _||$$  _$$\  _$$  _||$$
_$$\ 
$$ /  \_||$$ /  $$ |$$$$\  $$ |$$ |  $$ |$$ |  $$ |  $$ |  $$ |  $$
|  $$ |
$$ |  $$ |  $$ |$$ $$$\  $$$\  $$$ |$$$$$\  $$$$$$  |  $$ |  $$$$$\  $$
$$$$$  |
$$ |  $$ |  $$ |$$ \$$$$  | \$$\  $ /  $$  _||  $$  _$$<  $$ |  $$  _||  $$
_$$<
$$ |  $$\  $$ |  $$ |$$ |$$$$  | \$$$ /  $$ |  $$ |  $$ |  $$ |  $$
|  $$ |
\$$$$$  | $$$$$$  |$$ | \$$  |  \$ /  $$$$$$\  $$ |  $$ |  $$ |  $$$$$$\  $$
|  $$ |
 \_||  \_||  \_||  \_||  \_||  \_||  \_||  \_||  \_||  \_||  \_||  \_||
_||  \_||
""")
    time.sleep(1)
    while True:
        menu()

#Главное меню
def menu():
    print()
    print("Что вы хотите сделать?")
    print("1. Конвертировать")
```

```

print("2. Посмотреть историю")
print("0. Выйти")

try:
    choice = int(input(">> "))
    if choice == 1:
        convert_menu()
    elif choice == 2:
        history_menu()
    elif choice == 0:
        exit()
    else:
        print("Неверный ввод!")
except ValueError:
    print("Введите число!")

#Меню истории
def history_menu():
    while True:
        print("\nИстория конвертаций:")
        print("1. Последние 5 конвертаций")
        print("2. Вся история")
        print("3. Статистика")
        print("4. Очистить историю")
        print("0. Назад")
        choice = input(">> ")
        if choice == "1":
            show_recent_history()
        elif choice == "2":
            show_full_history()
        elif choice == "3":
            show_statistics()
        elif choice == "4":
            history.clear_history()
            print("История очищена.")
        elif choice == "0":
            return
        else:
            print("Неверный ввод!")

def show_recent_history():
    data = history.get_recent_conversions(5)
    if not data:
        print("История пуста.")
        return
    print("\nПоследние 5 конвертаций:")
    for entry in data:
        print(
            f"[{entry['timestamp']}] {entry['category']}: {entry['value']} "
            f"{entry['from_unit']} → {entry['result']} {entry['to_unit']}"
        )

def show_full_history():
    data = history.load_history()
    if not data:
        print("История пуста.")
        return
    print("\nВся история:")
    for entry in data:
        print(
            f"[{entry['timestamp']}] {entry['category']}: {entry['value']} "

```

```

{entry['from_unit']} → {entry['result']} {entry['to_unit']}")

def show_statistics():
    stats = history.get_statistics()
    if stats["total"] == 0:
        print("История пуста.")
        return
    print("\nСтатистика:")
    print(f"Всего конвертаций: {stats['total']}")
    print("По категориям:")
    for cat, count in stats["by_category"].items():
        print(f" - {cat}: {count}")
    print(f"Первая: {stats['first_conversion']}")
    print(f"Последняя: {stats['last_conversion']}")

#Меню выбора типа конвертации
def convert_menu():
    print()
    print("Что вы хотите конвертировать?")
    print("1. Длина")
    print("2. Температура")
    print("3. Масса")
    print("0. Назад")

    choice = input(">> ")
    if choice == "1":
        convert_linear_menu("длина")
    elif choice == "2":
        convert_temperature_menu()
    elif choice == "3":
        convert_linear_menu("масса")
    elif choice == "0":
        return
    else:
        print("Неверный ввод!")

#Конвертации длины/массы
def convert_linear_menu(category):
    print("\nДоступные единицы:")
    for unit in conversions.UNITS[category]:
        print(" -", unit)

    try:
        value_str = input("\nВведите значение: ")
        value = validator.validate_numeric_input(value_str)
        value = validator.validate_positive_number(value)

        from_unit = input("Из (например, метр): ").strip().lower()
        to_unit = input("В (например, километр): ").strip().lower()

        validator.validate_unit_exists(category, from_unit)
        validator.validate_unit_exists(category, to_unit)

        result = conversions.convert_linear(category, value, from_unit, to_unit)
        formatted_result = unit_formatter.format_conversion_result(value, from_unit,
result, to_unit)
        print(f"\n{formatted_result}\n")

        history.add_conversion(category, value, from_unit, to_unit, result)

```

```

except Exception as e:
    print("Ошибка:", e)

#Конвертации температур
def convert_temperature_menu():
    print("""
1. Цельсий --> Фаренгейт
2. Фаренгейт --> Цельсий
3. Цельсий --> Кельвин
4. Кельвин --> Цельсий
5. Фаренгейт --> Кельвин
6. Кельвин --> Фаренгейт
0. Назад
""")
    try:
        str_choice = input(">> ")
        choice = validator.validate_numeric_input(str_choice)

        if choice == 0:
            return

        str_value = input("Введите значение: ")
        value = validator.validate_numeric_input(str_value)

        conversions_map = {
            1: ("цельсий", "фаренгейт", conversions.f_to_c),
            2: ("фаренгейт", "цельсий", conversions.c_to_f),
            3: ("цельсий", "кельвин", conversions.c_to_k),
            4: ("кельвин", "цельсий", conversions.k_to_c),
            5: ("фаренгейт", "кельвин", conversions.f_to_k),
            6: ("кельвин", "фаренгейт", conversions.k_to_f)
        }

        if choice not in conversions_map:
            print("Неверный выбор!")
            return

        from_unit, to_unit, func = conversions_map[choice]
        value = validator.validate_temperature_range(value, from_unit[0])
        result = func(value)

        formatted_result = unit_formatter.format_conversion_result(value, from_unit,
result, to_unit)
        print(f"\n{formatted_result}\n")

        history.add_conversion("температура", value, from_unit, to_unit, result)

    except ValueError:
        print("Ошибка: введите число!")

if __name__ == "__main__":
    startup()

```

2.2 Модульное тестирование

Описание тестов: Тестовый набор охватывает функциональность

пользовательского интерфейса приложения, включая навигацию по меню, конвертацию линейных и температурных величин, работу с историей конвертаций и обработку ошибок валидации входных данных.

Листинг 14 — Код тестов.

```
import pytest
import sys
import os
import core.view as view
from unittest.mock import patch, MagicMock

@patch('builtins.input')
@patch('builtins.print')
def test_menu_navigation(mock_print, mock_input):
    mock_input.return_value = '1'
    with patch('core.view.convert_menu') as mock_convert:
        view.menu()
        mock_convert.assert_called_once()

@patch('builtins.input')
@patch('builtins.print')
def test_menu_invalid_input(mock_print, mock_input):
    mock_input.return_value = 'abc'
    view.menu()
    mock_print.assert_any_call("Введите число!")

@patch('builtins.input')
@patch('builtins.print')
def test_convert_menu_linear(mock_print, mock_input):
    mock_input.return_value = '1'
    with patch('core.view.convert_linear_menu') as mock_linear:
        view.convert_menu()
        mock_linear.assert_called_with('длина')

@patch('builtins.input')
@patch('builtins.print')
def test_convert_linear_menu_success(mock_print, mock_input):
    mock_input.side_effect = ['100', 'сантиметр', 'метр']

    with patch('core.validator.validate_numeric_input') as mock_val_num, \
        patch('core.validator.validate_positive_number') as mock_val_pos, \
        patch('core.validator.validate_unit_exists') as mock_val_unit, \
        patch('core.conversions.convert_linear') as mock_convert, \
        patch('core.unit_formatter.format_conversion_result') as mock_format, \
        patch('core.history.add_conversion') as mock_history:

        mock_val_num.return_value = 100.0
        mock_val_pos.return_value = 100.0
        mock_convert.return_value = 1.0
        mock_format.return_value = "100 см = 1 м"

        view.convert_linear_menu('длина')

        mock_convert.assert_called_once()
        mock_history.assert_called_once()
        mock_print.assert_any_call("\n100 см = 1 м\n")

@patch('builtins.input')
```



```

@patch('builtins.print')
def test_convert_temperature_menu_success(mock_print, mock_input):
    mock_input.side_effect = ['1', '0']

    with patch('core.validator.validate_numeric_input') as mock_val_num, \
        patch('core.conversions.c_to_f') as mock_convert, \
        patch('core.unit_formatter.format_conversion_result') as mock_format, \
        patch('core.history.add_conversion') as mock_history:

        mock_val_num.side_effect = [1.0, 0.0]
        mock_convert.return_value = 32.0
        mock_format.return_value = "0°C = 32°F"

        view.convert_temperature_menu()

        mock_convert.assert_called_once_with(0.0)
        mock_history.assert_called_once()

@patch('core.history.get_recent_conversions')
@patch('builtins.print')
def test_show_recent_history(mock_print, mock_get_recent):
    mock_get_recent.return_value = [{
        'timestamp': '2024-01-15T10:00:00',
        'category': 'длина',
        'value': 100.0,
        'from_unit': 'см',
        'to_unit': 'м',
        'result': 1.0
    }]

    view.show_recent_history()
    mock_print.assert_any_call("\nПоследние 5 конвертаций:")

@patch('core.history.get_statistics')
@patch('builtins.print')
def test_show_statistics(mock_print, mock_get_stats):
    mock_get_stats.return_value = {
        'total': 5,
        'by_category': {'длина': 3, 'масса': 2},
        'first_conversion': '2024-01-15T10:00:00',
        'last_conversion': '2024-01-15T11:00:00'
    }

    view.show_statistics()
    mock_print.assert_any_call("Всего конвертаций: 5")

@patch('builtins.input')
@patch('builtins.print')
def test_history_menu_clear(mock_print, mock_input):
    mock_input.side_effect = ['4', '0']

    with patch('core.history.clear_history') as mock_clear:
        view.history_menu()
        mock_clear.assert_called_once()
        mock_print.assert_any_call("История очищена.")

@patch('builtins.input')
@patch('builtins.print')
def test_convert_linear_menu_validation_error(mock_print, mock_input):
    mock_input.side_effect = ['invalid', 'см', 'м']

```

```

with patch('core.validator.validate_numeric_input') as mock_val_num:
    mock_val_num.side_effect = ValueError("Ошибка числа")

    view.convert_linear_menu('длина')
    mock_print.assert_any_call("Ошибка:", "Ошибка числа")

@patch('builtins.input')
@patch('builtins.print')
def test_convert_temperature_invalid_choice(mock_print, mock_input):
    mock_input.side_effect = ['7', '0']

    with patch('core.validator.validate_numeric_input') as mock_val_num:
        mock_val_num.return_value = 7.0

        view.convert_temperature_menu()
        mock_print.assert_any_call("Неверный выбор!")

```

Методология: Тестирование реализовано с использованием моков и заглушек для имитации пользовательского ввода и вывода, а также для изоляции тестируемого кода от внешних зависимостей. Проверяется как корректная работа интерфейса, так и обработка ошибочных ситуаций.

Анализ покрытия кода: Тесты охватывают все основные компоненты пользовательского интерфейса: главное меню, меню конвертации, обработку различных типов конвертаций, работу с историей и статистикой. Проверяется корректность отображения результатов, обработка ошибок валидации и взаимодействие с другими модулями приложения. Особое внимание уделяется тестированию граничных случаев и обработке некорректного ввода.

Исправление ошибок:

Краткое описание ошибки: «Неверное преобразование Цельсия в Фаренгейты».

Статус ошибки: открыта («Open»).

Категория ошибки: серьезная («Major»).

Тестовый случай: «Проверка алгоритма функционирования программы».

Описание ошибки:

1. Загрузить программу.

2. В поле ввода ввести строку «1».
2. В поле ввода ввести строку «2».
2. В поле ввода ввести строку «1».
2. В поле ввода ввести строку «10».
2. В поле ввода ввести строку «0».
2. В поле ввода ввести строку «0».
2. В поле ввода ввести строку «2».
2. В поле ввода ввести строку «1».
4. Полученный результат: «-12.22». Ожидаемый результат: «50».

Листинг 15 — Исправленный код.

```
import time
import core.convert_script as conversions
import core.validator as validator
import core.unit_formatter as unit_formatter
import core.history as history

def startup():
    print(r"""
/$$$$$\  $$$$$\ $$\  $$\  $$\  $$$\ $$$$$$$\ $$$$$$$\ $$$$$$$\ $$$$
$$$$$\ 
$$ _$$\ $$ _$$\ $$$\ $$ |$$ |  $$ |$$ _||$$ _$$\__$$ _||$$ _||$$
_$$\ 
$$ /  \_||$$ /  $$ |$$$$\ $$ |$$ |  $$ |$$ |  $$ |  $$ |  $$ 
|  $$ |
$$ |  $$ |  $$ |$$ $$$\ $$ |\$$\  $$ |$$$$$\  $$$$$$  |  $$ |  $$$$$\  $$
$$$$$  |
$$ |  $$ |  $$ |$$ \$$$$ | \$$\$$ /  $$ _|  $$ _$$<  $$ |  $$ _|  $$
_$$<
$$ |  $$\  $$ |  $$ |$$ |\$$$ | \$$$ /  $$ |  $$ |  $$ |  $$ |  $$ 
|  $$ |
\$$$$$  | $$$$$$ |$$ | \$$ |  \$ /  $$$$$$$\ $$ |  $$ |  $$ |  $$$$$$$\ $$
|  $$ |
| \_||  \_||  \_||  \_||  \_||  \_||  \_||  \_||  \_||  \_||  \_||
_||  \_||
""")
    time.sleep(1)
    while True:
        menu()

#Главное меню
def menu():
    print()
    print("Что вы хотите сделать?")
    print("1. Конвертировать")
    print("2. Посмотреть историю")
    print("0. Выйти")
```

```

try:
    choice = int(input(">> "))
    if choice == 1:
        convert_menu()
    elif choice == 2:
        history_menu()
    elif choice == 0:
        exit()
    else:
        print("Неверный ввод!")
except ValueError:
    print("Введите число!")

#Меню истории
def history_menu():
    while True:
        print("\nИстория конвертаций:")
        print("1. Последние 5 конвертаций")
        print("2. Вся история")
        print("3. Статистика")
        print("4. Очистить историю")
        print("0. Назад")
        choice = input(">> ")
        if choice == "1":
            show_recent_history()
        elif choice == "2":
            show_full_history()
        elif choice == "3":
            show_statistics()
        elif choice == "4":
            history.clear_history()
            print("История очищена.")
        elif choice == "0":
            return
        else:
            print("Неверный ввод!")

def show_recent_history():
    data = history.get_recent_conversions(5)
    if not data:
        print("История пуста.")
        return
    print("\nПоследние 5 конвертаций:")
    for entry in data:
        print(
            f"[{entry['timestamp']}] {entry['category']}: {entry['value']} "
            f"{entry['from_unit']} → {entry['result']} {entry['to_unit']}")

def show_full_history():
    data = history.load_history()
    if not data:
        print("История пуста.")
        return
    print("\nВся история:")
    for entry in data:
        print(
            f"[{entry['timestamp']}] {entry['category']}: {entry['value']} "
            f"{entry['from_unit']} → {entry['result']} {entry['to_unit']}")

```

```

def show_statistics():
    stats = history.get_statistics()
    if stats["total"] == 0:
        print("История пуста.")
        return
    print("\nСтатистика:")
    print(f"Всего конвертаций: {stats['total']}")
    print("По категориям:")
    for cat, count in stats["by_category"].items():
        print(f" - {cat}: {count}")
    print(f"Первая: {stats['first_conversion']}")
    print(f"Последняя: {stats['last_conversion']}")

#Меню выбора типа конвертации
def convert_menu():
    print()
    print("Что вы хотите конвертировать?")
    print("1. Длина")
    print("2. Температура")
    print("3. Масса")
    print("0. Назад")

    choice = input(">> ")
    if choice == "1":
        convert_linear_menu("длина")
    elif choice == "2":
        convert_temperature_menu()
    elif choice == "3":
        convert_linear_menu("масса")
    elif choice == "0":
        return
    else:
        print("Неверный ввод!")

#Конвертации длины/массы
def convert_linear_menu(category):
    print("\nДоступные единицы:")
    for unit in conversions.UNITS[category]:
        print(" -", unit)

    try:
        value_str = input("\nВведите значение: ")
        value = validator.validate_numeric_input(value_str)
        value = validator.validate_positive_number(value)

        from_unit = input("Из (например, метр): ").strip().lower()
        to_unit = input("В (например, километр): ").strip().lower()

        validator.validate_unit_exists(category, from_unit)
        validator.validate_unit_exists(category, to_unit)

        result = conversions.convert_linear(category, value, from_unit, to_unit)
        formatted_result = unit_formatter.format_conversion_result(value, from_unit,
result, to_unit)
        print(f"\n{formatted_result}\n")

        history.add_conversion(category, value, from_unit, to_unit, result)

    except Exception as e:
        print("Ошибка:", e)

```

```

#Конвертации температур
def convert_temperature_menu():
    print("""
1. Цельсий --> Фаренгейт
2. Фаренгейт --> Цельсий
3. Цельсий --> Кельвин
4. Кельвин --> Цельсий
5. Фаренгейт --> Кельвин
6. Кельвин --> Фаренгейт
0. Назад
""")
    try:
        str_choice = input(">> ")
        choice = validator.validate_numeric_input(str_choice)

        if choice == 0:
            return

        str_value = input("Введите значение: ")
        value = validator.validate_numeric_input(str_value)

        conversions_map = {
            1: ("цельсий", "фаренгейт", conversions.c_to_f),
            2: ("фаренгейт", "цельсий", conversions.f_to_c),
            3: ("цельсий", "кельвин", conversions.c_to_k),
            4: ("кельвин", "цельсий", conversions.k_to_c),
            5: ("фаренгейт", "кельвин", conversions.f_to_k),
            6: ("кельвин", "фаренгейт", conversions.k_to_f)
        }

        if choice not in conversions_map:
            print("Неверный выбор!")
            return

        from_unit, to_unit, func = conversions_map[choice]
        value = validator.validate_temperature_range(value, from_unit[0])
        result = func(value)

        formatted_result = unit_formatter.format_conversion_result(value, from_unit,
result, to_unit)
        print(f"\n{formatted_result}\n")

        history.add_conversion("температура", value, from_unit, to_unit, result)

    except ValueError:
        print("Ошибка: введите число!")

if __name__ == "__main__":
    startup()

```

2.3 Мутационное тестирование

Создание мутантов:

1. Мутации в menu и history_menu (Управление потоком)

Функция	Исходный код	Мутант (Ошибка)	Результат теста	Убит ?	Недостаток теста?
menu	<pre>if choice == 1:</pre>	<pre>if choice != 1: (Инверсия условия)</pre>	<pre>test_menu_navigation Провалится (не вызовет convert_menu)</pre>	 Да	Нет
menu	<pre>try: choice = int(input("> "))</pre>	<pre>try: choice = input("> ") (Удаление int())</pre>	<pre>test_menu_navigation Провалится (choice будет строкой, а не числом)</pre>	 Да	Нет
history_menu	<pre>elif choice == "0": return</pre>	<pre>elif choice == "0": break (Изменение оператора выхода)</pre>	Выживет (оба оператора прервут while True цикл)	 Нет	Эквивалентный мутант
history_menu	<pre>if not data: print("История пуста.")</pre>	<pre>if data: print("История пуста.") (Инверсия условия)</pre>	<pre>test_show_recent_history (должен печатать историю, но вместо этого напечатает "История пуста.") Провалится</pre>	 Да	Нет

2. Мутации в convert_linear_menu

Исходный код	Мутант (Ошибка)	Результат теста	Убит ?	Недостаток теста?
<code>value = validator.validate_positive_number(value)</code>	Удаление строки (Удаление валидации)	Выживет	 Нет	Да
<code>validator.validate_unit_exists(category, to_unit)</code>	Удаление строки (Удаление валидации)	Выживет	 Нет	Да
<code>history.add_conversion(...)</code>	Удаление строки (Удаление побочного эффекта)	Выживет	 Нет	Да
<code>except Exception as e: print("Ошибка:", e)</code>	<code>except ValueError as e: print("Ошибка:", e)</code> (Сужение типа исключения)	<code>test_convert_linea_menu_validation_error</code> Выживет (в тесте бросается <code>ValueError</code>)	 Нет	Да

3. Мутации в `convert_temperature_menu`

Исходный код	Мутант (Ошибка)	Результат теста	Убит ?	Недостаток теста?
<code>if choice not in conversions_map:</code>	<code>if choice in conversions_map:</code> (Инверсия условия)	<code>test_convert_temperature_invalid_choice</code> Провалится (вызовет ошибку <code>KeyError</code> вместо)	 Да	Нет

Исходный код	Мутант (Ошибка)	Результат теста	Убит ?	Недостаток теста?
		print("Неверный выбор!"))		
value = validator.validate_temperature_range(value, from_unit[0])	Удаление строки (Удаление валидации)	Выживет	 Нет	Да
from_unit[0]	from_unit[1] (Изменение индекса)	Выживет	 Нет	Да

Анализ их выживания:

Наш тест `test_convert_linear_menu_success` проверяет, что функции вызываются (`assert_called_once`), но неявно предполагает, что они успешны. Мутанты, удаляющие важные вызовы валидации или записи в историю, выживают, потому что:

1. Удаление `validator.validate_positive_number`: Тест использует `value=100`, что является положительным числом, и не проверяет вход с отрицательным числом.
2. Удаление `validator.validate_unit_exists`: Тест использует корректные единицы. Мутант выживает, потому что тест не проверяет, что при неверной единице должна произойти ошибка.

Обнаруженные недостатки тестов в `convert_temperature_menu`:

1. Удаление `validator.validate_temperature_range`: Тест `test_convert_temperature_menu_success` использует `value=0`, что находится в безопасном диапазоне. Мутант, удаляющий эту валидацию, выживает, потому что тест не включает входное значение, которое находится вне диапазона (например, `-300C`).

2. Изменение индекса `from_unit[0]`: При использовании 'цельсий' (с - 0) или 'фаренгейт' (f - 0) в качестве `from_unit`, изменение индекса на [1] приведет к ошибке, только если второй символ не является ожидаемой шкалой. В нашем коде `from_unit` — это полные слова ('цельсий', 'фаренгейт' и т.д.). Использование `from_unit[0]` (первой буквы) для проверки диапазона очень хрупкое. Если мутант изменит это на `from_unit[1]`, тест может выжить, если `from_unit[1]` по совпадению соответствует ключу.

Корректировка тестов:

1. Чтобы убить выживших мутантов в `convert_linear_menu` (Валидация), мы проверили, что ввод отрицательного числа или несуществующей единицы вызывает ошибку, и что функция корректно ее обрабатывает (except).

Листинг 25 — Код теста.

```
@patch('builtins.input')
@patch('builtins.print')
def test_convert_linear_menu_negative_value(mock_print, mock_input):
    # Ввод отрицательного числа
    mock_input.side_effect = ['-10', 'метр', 'километр']

    # Мы знаем, что validate_positive_number выдаст ValueError
    with patch('core.validator.validate_positive_number',
side_effect=ValueError("Ошибка: значение должно быть положительным")):
        view.convert_linear_menu('длина')
        # Проверяем, что была напечатана ошибка
        mock_print.assert_any_call("Ошибка:", 'Ошибка: значение должно быть
положительным') # Убивает мутанта с удаленной валидацией

@patch('builtins.input')
@patch('builtins.print')
def test_convert_linear_menu_invalid_unit(mock_print, mock_input):
    # Ввод существующего числа, но неверной единицы
    mock_input.side_effect = ['100', 'фут', 'метр']

    with patch('core.validator.validate_unit_exists',
side_effect=ValueError("Единица 'фут' не найдена")):
        view.convert_linear_menu('длина')
        # Проверяем, что была напечатана ошибка
        mock_print.assert_any_call("Ошибка:", "Единица 'фут' не найдена") #
Убивает мутанта с удаленной валидацией
```

2. Чтобы убить выжившего мутанта в `convert_temperature_menu` (Валидация), мы добавили проверку, что ввод температуры вне диапазона вызывает ошибку, и что эта ошибка также корректно обрабатывается.

Листинг 26 — Код теста.

```
@patch('builtins.input')
@patch('builtins.print')
def test_convert_temperature_menu_out_of_range(mock_print, mock_input):
    # Выбор Цельсий->Фаренгейт (1), ввод -300C (вне диапазона)
    mock_input.side_effect = ['1', '-300']

    with patch('core.validator.validate_temperature_range',
side_effect=ValueError("Температура должна быть в диапазоне")):
        view.convert_temperature_menu()
        # Проверяем, что была напечатана ошибка
        mock_print.assert_any_call("Ошибка:", 'Температура должна быть в
диапазоне') # убивает мутанта с удаленной валидацией
```

3 АНАЛИЗ И ВЫВОДЫ

3.1 Оценка качества тестирования

Итоговое Качество Тестирования:

1. Полнота (Покрытие): Теперь тесты обеспечивают не только высокое покрытие строк кода, но и высокое покрытие логики. Почти все значимые операторы (математические, логические, сравнения) проверяются на их чувствительность к ошибкам.
2. Надежность (Чувствительность): Набор тестов способен обнаруживать даже атомарные, синтаксически небольшие ошибки (мутанты), которые могли бы быть допущены программистом, что является прямым доказательством его надежности.
3. Тестирование интеграционных точек: Особое внимание уделено интеграционным точкам (например, view вызывает validator и history). Проверка, что удаление валидации приводит к сбою, а удаление записи в историю обнаруживается, делает интеграционные тесты более надежными.

3.2 Анализ недостатков и их устранение

Улучшения, которые были достигнуты благодаря мутационному тестированию:

Модуль	Ранее Выжившие Мутанты (Пробелы)	Статус после изменений
view	Критический пробел: не проверялась корректная обработка исключений валидации.	Убиты. Добавлены тесты, имитирующие некорректный ввод (отрицательные числа, несуществующие единицы) и подтверждающие, что: 1) вызывается ошибка; 2) выводится сообщение об ошибке; 3) конвертация и запись в историю не происходят.

3.3 Итоговые выводы по результатам работы

В результате выполнения практической работы были достигнуты поставленные цели: освоены методы модульного и мутационного тестирования, разработаны и проанализированы тестовые наборы. В ходе исследования успешно применено мутационное тестирование, созданы эффективные модульные тесты с высоким покрытием кода. Мутационное тестирование позволило выявить и устранить пробелы в тестовом наборе, повысив качество проверки программного продукта. Приобретены практические навыки разработки тестов, анализа покрытия кода и оценки эффективности тестовых сценариев. Полученные результаты подтверждают, что комбинированный подход к тестированию обеспечивает надёжную проверку качества программного обеспечения и помогает обнаруживать потенциальные ошибки на ранних этапах разработки.